实验 2 操作手册：寄存器（EGO1）

适用资料：2026-10-20 网站快照与本目录的《lab2-寄存器.pdf》。本文是实验操作指南，不是待提交的实验报告。以下板上现象均为**待验证的预期结果**；照片、截图及结论必须来自你自己的实验。

第 1 步：准备好文件，先分清两个工程

目的：做完 D 触发器，再做寄存器文件，不把两个工程混在一起。

准备 EGO1、可传输数据的 USB 线、装有 Vivado 的实验室电脑、学生卡，以及整个 `lab2/code/` 文件夹。器件选择 `xc7a35tcsg324-1`，不是 Minisys 的器件。

工程名	设计文件	仿真文件	约束文件
dff	dff/dff.v	dff/dff_sim.v	dff/pin.xdc
reg8file	reg8file/reg8file.v	reg8file/testbench.v	reg8file/pin.xdc

1. 在电脑上建一个纯英文工作目录，例如 `D:/FPGA/lab2/`，把源码复制到其中的 `code/`。

2. 把 Vivado 工程分别建在 `D:/FPGA/lab2/projects/dff/` 和 `D:/FPGA/lab2/projects/reg8file/`，不要建进源码目录。

3. 另建 `evidence/` 文件夹，存放截图和实物照片。这里不放预制或模拟的“实验结果图”。

4. 先通读后面的开关映射。两个工程虽然都叫 `pin.xdc`，内容并不相同。

必须分清：

• Design Sources：可综合的电路代码，只添加当前工程的 RTL 文件。

• Simulation Sources：产生输入激励的测试代码，不能加入 Design Sources。

• Constraints：当前工程的那一份 XDC。不要同时添加两个 `pin.xdc`。

• `dff_check.v` 和 `reg8file_check.v`：辅助本地自检，正常做实验时不必导入。

• `.vcd`：本地仿真波形，不是 FPGA 下载文件。上板要用 Vivado 生成的 `.bit`。

本次最终要留的材料： 寄存器文件 3 张实物照片、寄存器文件仿真波形、D 触发器与寄存器文件各 2 张电路图，共 4 张 Schematic。D 触发器也要上板自行验证，但当前报告清单没有要求它的实物照片。

检查点： 你手上有两套设计/仿真/约束文件，且知道哪一套属于哪个工程。不要照搬 Lab1 的 mux 文件或约束。

第 2 步：建立 dff 工程，先运行行为仿真

目的：确认清零、写入、保持的逻辑，再去综合和上板。

1. 打开 Vivado，点击 **Create Project → Next**。

2. Project name 填 `dff`，选择工作目录，保留 **Create project subdirectory**。

3. Project Type 选 **RTL Project**；可勾选 **Do not specify sources at this time**，之后统一添加。

4. Default Part 选 **Parts**，搜索并选中 `xc7a35tcsg324-1`，核对后 Finish。若搜索不到，请检查是否安装 Artix-7 器件支持，不要随便选一个型号。

5. 点击 **Add Sources → Add or create design sources → Add Files**，只添加 `code/dff/dff.v`，Finish。

6. 再点 **Add Sources → Add or create simulation sources → Add Files**，添加 `code/dff/dff_sim.v`。

7. 在 Sources 中展开 Design Sources，右击 `dff` → **Set as Top**；展开 Simulation Sources，右击 `dff_sim` → **Set as Top**。已是顶层时菜单可能不可选。

8. 点击 **Run Simulation → Run Behavioral Simulation**。若未跑完，点击 Run All，仿真在 100 ns 停止。

9. 在波形窗口保留 `clk`、`clr`、`en`、`d`、`q`，点击 **Zoom Fit** 显示 0 – 100 ns。缺信号时，在 Scope 选 `dff_sim`，从 Objects 选中信号，右击 Add to Wave Window，再 Restart、Run All。

对照波形看什么

时间 / ns	输入操作	q 的预期行为
0 – 10	clr=1	清零后为 0
10	clr=0，en=1，d=1	还未到上升沿，q 仍为 0
15	时钟上升沿	q 变为 1
20 / 25	d 先变 0；随后上升沿	q 到 25 ns 才变 0
30 / 35	d 先变 1；随后上升沿	q 到 35 ns 才变 1
40 – 50	en=0，d=0	45 ns 上升沿也不更新，q 保持 1
50 – 100	clr=1	q 在 50 ns 清零，不等 55 ns 上升沿

`dff_sim.v` 的激励来自官网原始示例，仅重命名仿真模块并增加可选波形导出。**它不会打印自检 PASS，不能因未看到 PASS 就判断失败。**

检查点： q 只在允许写入的上升沿采样；复位不等时钟；禁用后保持。输入变化安排在下降沿，避免与上升沿竞争。这里的 100 MHz 时钟是测试代码模拟的时钟，上板时才来自 P17 晶振。

第 3 步：保存 dff 电路图，下载并验证

目的：保存报告需要的两张电路图，并实际检查 D 触发器。

先保存 RTL，再添加约束和综合

1. 关闭仿真，回到 Flow Navigator。点击 **RTL Analysis → Open Elaborated Design**，进入后打开 **Schematic**。

2. 选中顶层模块，必要时展开节点，调整缩放，让输入输出与触发器结构可读。截取 Schematic，而不是代码编辑器，保存 `evidence/dff_rtl.png`。

3. **Add Sources → Add or create constraints → Add Files**，只添加 `code/dff/pin.xdc`。核对如下映射：

信号	板上器件	管脚
clk	板载 100 MHz 时钟	P17
clr	普通按键 S1	R17
en	大拨码 SW7	P5
d	大拨码 SW0	R1
q	LD2_0	K2

4. 点击 **Run Synthesis**，完成后 **Open Synthesized Design → Schematic**。保存 `evidence/dff_synth.png`，不能用 Device 布局图代替。

5. 点击 **Run Implementation**，完成后 **Generate Bitstream**。有 Errors 或 Critical Warnings 时先看 Messages，定位并处理，不靠降低 DRC 等级强行通过。

6. 连接 EGO1 的 USB-UART/JTAG 接口并按实验室要求供电。打开 **Open Hardware Manager → Open Target → Auto Connect**。

7. 选中 FPGA，点击 **Program Device**，选择本工程 `dff.runs/impl_1/dff.bit`，点击 Program。确认不是之前的 mux.bit。

按顺序拨开关

1. SW7、SW0 都向下，按一下普通按键 S1 后松开；预期 LD2_0 熄灭。

2. SW7 向上打开使能。SW0 向上，预期 LD2_0 亮；SW0 向下，预期灭。100 MHz 太快，肉眼看起来像立即跟随，不会看到每个时钟。

3. 把 SW0 向上，等灯亮，再把 SW7 向下关闭使能。

4. 来回拨 SW0，预期灯一直亮。保持阶段没有写入，因此改变 d 不影响 q。

5. 保持 SW7 向下，按一下 S1 并松开，预期灯灭，之后继续改变 SW0 仍不亮。

检查点：“使能跟随、禁用保持、按键清零”都验证过；两张 D 触发器电路图已经保存。清零测试先关使能，否则松开 S1 后可能立即写回 d=1，看起来像没有清零。

不要按错： S1 是 R17 对应的普通按键，不是 S5/PROG，也不是 S6/RST。按 PROG 可能清除当前 FPGA 配置，需要重新下载。

第 4 步：记住寄存器文件的开关布局

目的：用三组开关分别控制数据、写地址和读地址。

寄存器文件有 0 – 7 共 8 个寄存器，每个存 8 位。`wsel` 决定写哪个，`rsel` 决定看哪个，两个地址互相独立。**q 显示的是 rsel 选中的内容，不是自动显示刚写入的内容。**

功能	板上器件	从高位到低位读
写使能 en	SW7	向上 1，向下 0
未使用	SW6	保持向下
读地址 rsel[2:0]	SW5、SW4、SW3	SW5 为最高位
写地址 wsel[2:0]	SW2、SW1、SW0	SW2 为最高位
输入 d[7:0]	DIP7⋯DIP0	DIP7 为最高位
输出 q[7:0]	LD2_7⋯LD2_0	LD2_7 为最高位
清零 clr	S1 / R17	按下为 1，松开为 0
时钟 clk	P17	板载 100 MHz，不用手动产生

按管脚识别 DIP，别按外壳数字猜位号

位号	DIP / d 管脚	LD2 / q 管脚
0	T5	K2
1	T3	J2
2	R3	J3
3	V4	H4
4	V5	J4
5	V2	G3
6	U2	G4
7	U3	F6

这里的 DIP0 是 T5，不是大拨码 SW0/R1。塑料外壳可能标 1 – 8，本手册使用数据位 0 – 7，以 FPGA 管脚丝印为准。按课程说明，开关上拨为 1、下拨为 0；若板子摆放方向不同，先按丝印确认，不凭照片中的左右位置判断。

本次统一用两个非零数据：**R1 = 0x3C = 00111100**，**R6 = 0xA5 = 10100101**。以下 `R1`、`R6` 指寄存器编号，不是 FPGA 管脚名称。

最重要的规则：先关 SW7 → 改地址和数据 → 开 SW7 写入 → 再关 SW7。 SW7 打开时，每秒有一亿次写入机会；不要开着使能慢慢拨地址，否则可能污染经过的寄存器。

检查点： 能明确说出 SW7 是写使能、SW2⋯SW0 是写地址、SW5⋯SW3 是读地址，DIP 是数据。SW6 没有功能，不需要约束它。

第 5 步：建立 reg8file 工程并保存仿真波形

目的：展示“初始复位 → 两次写入 → 两次读取 → 异步清零”。

1. **File → Project → New**，另建工程 `reg8file`，器件仍为 `xc7a35tcsg324-1`。不要在 dff 工程上继续堆文件。

2. 添加 `code/reg8file/reg8file.v` 到 Design Sources，设设计顶层为 `reg8file`。

3. 添加 `code/reg8file/testbench.v` 到 Simulation Sources，设仿真顶层为 `testbench`。

4. **Run Simulation → Run Behavioral Simulation**，Run All，仿真在 100 ns 结束。控制台应有 `PASS: testbench, 10 checks`，不得有 FATAL。

5. 波形顺序调整为 `clk`、`clr`、`en`、`wsel`、`rsel`、`d`、`q`。`wsel`/`rsel` 右击 Radix 选 Unsigned Decimal；`d`/`q` 选 Hexadecimal。

6. 点击 Zoom Fit，显示 0 – 100 ns。放大信号名区域，确保 d/q 的总线值、时间刻度清晰。不要用辅助长时间自检的波形代替。

时间 / ns	操作	预期结果
0 – 10	clr=1，en=0	全部清零，读 R0 得 00
10 / 15	en=1，wsel=1，d=3C	15 ns 上升沿写入 R1；rsel=0，q 仍为 00
20 – 30	en=0，d=00	不覆盖 R1
30 / 35	en=1，wsel=6，d=A5	35 ns 上升沿写入 R6；q 仍读 R0 为 00
40	en=0，wsel=0，d=00	停止写入，后续只读取
42	rsel=1	q 变为 3C，不等 45 ns 上升沿
52	rsel=6	q 变为 A5，不等 55 ns 上升沿
62	clr=1	q 立即变为 00，不等 65 ns 上升沿
70 / 72	释放 clr；rsel 改为 1	R1 也为 00
82 – 100	rsel=6	R6 仍为 00

写入用上升沿；读取和清零故意安排在 42、52、62 ns，以明确区分同步与异步。表中时刻均指本次提供的 `testbench.v`，不要混用 D 触发器示例的时间。

截图： 保存 `evidence/reg8file_wave.png`。只截信号名、波形和时间刻度，别截整屏代码窗口。若一张看不清，可另补 40 – 85 ns 的局部图，但必须保留完整流程图。

分析时要写： 15/35 ns 的写入条件、42/52 ns 的异步读、62 ns 的异步清零，以及清零后另一个已写寄存器也为零。按自己的真实截图描述，不把此表当成实测记录。

检查点： 两个不同寄存器写入了不同数据，波形显示两次读取和复位后读取，且保存了清晰截图。

第 6 步：保存电路图、下载，再写入两组数据

目的：完成寄存器文件硬件实现，为三张实物照片准备数据。

1. 关闭仿真，**Open Elaborated Design → Schematic**，保存 evidence/reg8file_rtl.png。

2. 添加且只添加 code/reg8file/pin.xdc 到 Constraints。确认 clk=P17、clr=R17，时钟周期为 10 ns。

3. **Run Synthesis → Open Synthesized Design → Schematic**，保存 evidence/reg8file_synth.png。官网只要求综合图整体截一张，不必放大后拼多张。

4. **Run Implementation → Generate Bitstream**。检查错误、关键警告及 Timing Summary；有负裕量或未约束路径先核对原因，不能把“生成 bit”自动等同于时序全部通过。

5. 打开 Hardware Manager，Auto Connect，Program Device，选 reg8file.runs/impl_1/reg8file.bit。确认这次已替换之前的 dff.bit。

6. 所有大拨码和 DIP 先拨为 0，按一下 S1 后松开。预期当前读取的 R0 为零，LD2 全灭。

写入第一组：R1 = 3C

1. 确认 SW7 向下。

2. 设置写地址 SW2 SW1 SW0 = 0 0 1，也就是只有 SW0 向上。

3. 设置 DIP7…DIP0 = 00111100：把 DIP2、3、4、5 向上，其他向下。

4. 把 SW7 向上，停留片刻，再向下。地址和数据保持不动，数据已在时钟上升沿写入。

5. 此时读地址仍为 0，LED 可能全灭，这是正常的，不代表没写进去。

写入第二组：R6 = A5

1. 再次确认 SW7 向下，才开始改开关。

2. 设置写地址 SW2 SW1 SW0 = 1 1 0，也就是 SW2、SW1 向上，SW0 向下。

3. 设置 DIP7…DIP0 = 10100101：把 DIP7、5、2、0 向上，其他向下。

4. 把 SW7 向上，停留片刻，再向下。后续读数和拍照时一直保持关闭。

5. 为让照片更容易说明，可在 SW7 关闭时将 DIP 全拨为 0，写地址也改为 000；存储的数据不应改变。

检查点： 两次写入都执行了“关闭 → 配置 → 打开 → 关闭”。现在 en=0、d=00、wsel=0；R1、R6 应分别保存 3C、A5。

第 7 步：读取、清零，拍齐三张照片

目的：按官网要求留“两次不同读取 + 清零后再读取”的三张图。

把学生卡放在开发板旁，确保每张照片能读到姓名或学号，能看清开关和 LD2。不要只拍灯的特写而丢掉输入状态。三次拍照时 SW7 都向下，S1 均已松开。

照片 1：读取 R1

1. 保持 `en=0、wsel=000、d=00000000`。
2. 设置读地址 `SW5 SW4 SW3 = 0 0 1`，即只有 SW3 向上。
3. 预期 `LD2_7…LD2_0 = 00111100`，亮的是 **LD2_2、LD2_3、LD2_4、LD2_5**。
4. 观察稳定后拍照，保存 `evidence/read_r1.jpg`。

照片 2：读取 R6

1. 不动使能、写地址和数据，把读地址改成 `SW5 SW4 SW3 = 1 1 0`。
2. 预期 `LD2_7…LD2_0 = 10100101`，亮的是 **LD2_7、LD2_5、LD2_2、LD2_0**。
3. 观察稳定后拍照，保存 `evidence/read_r6.jpg`。

清零，然后照片 3：换回 R1 再读取

1. 保持 SW7 向下、当前仍读 R6，按一下 **S1/R17** 后松开，预期全部 LD2 熄灭。不需要拍按住按键的照片。
2. 把读地址换回 `SW5 SW4 SW3 = 0 0 1`，也就是读另一个之前写过的 R1。
3. 预期 `LD2_7…LD2_0 = 00000000`，全灭。保存 `evidence/read_r1_after_clear.jpg`。
4. 可额外切换其他地址检查均为零，不必为每个地址拍照。三张照片不能证明全部内部状态，只是课程要求的实物展示。

拍照时的统一核对表

照片	SW7…SW0	DIP7…DIP0	rsel	预期 q
读取 R1	00001000	00000000	001	00111100
读取 R6	00110000	00000000	110	10100101
清零后读 R1	00001000	00000000	001	00000000

此表假定按第 6 步将 wsel 和 d 都归零，SW6 始终为 0。SW7…SW0 的格式是 `en、未使用、rsel[2:0]、wsel[2:0]`，不是 8 位数据。

检查点： 三张图对应两个不同寄存器、两个不同非零值和复位后的零值。若实际结果不同，先排查，不要照着预期值写“验证成功”。

第 8 步：整理报告与提交文件

目的：材料齐全、不漏图、不把操作手册当报告。

报告标题为“实验2：寄存器”，填姓名、学号、班级。正文按当前官网要求组织：

1. **寄存器文件实验结果图**：先说明 en、wsel、rsel、d、q 对应的开关与 LED；放第 7 步的三张实物照片。每张图下说明实际输入取值、读到的寄存器和实际输出；说明第三张在清零之后拍摄。

2. **寄存器文件仿真波形截图和波形分析**：放实际 Vivado 波形。按初始复位、两次写入、两次读取、异步清零顺序分析，标明关键时间和信号值。

3. **电路图**：放 D 触发器 RTL 与 Synthesis 各一张；寄存器文件 RTL 与 Synthesis 各一张。综合图用 Schematic，不是 Device 图。寄存器文件综合图整体截一张即可。

官网模板在“三、电路图”下使用“2.1 D触发器”“2.2 寄存器文件”；若直接沿用模板，保留原编号，不擅自替用户改写模板。

导出报告 PDF 前逐项核对

- 三张实物照片中学生卡姓名或学号可读，开关状态和输出能看清。
- 两次清零前读取的地址不同、输出不同且非零；清零后换回另一个地址读取。
- 波形覆盖完整过程，有信号名、数值和时间刻度，没有截错仿真顶层。
- 4 张电路图均来自当前正确工程，电路主体可读。
- 中文宋体、布局简洁、图片等比缩放，不裁掉关键信息。
- 用 PDF 阅读器重新打开导出文件，逐页检查缺字、空白页和溢出。

ZIP 内只放课程要求的内容

```
dff/
  dff.v
  pin.xdc
reg8file/
  reg8file.v
  pin.xdc
  testbench.v
学号_姓名.pdf
```

`xx.xdc` 是官网的泛指文件名，这里使用 `pin.xdc` 即可。**不要放整个 Vivado 工程、build、VCD、自检脚本或本操作手册 PDF。** D 触发器仿真文件保留在本地，当前提交清单没有要求它。

检查点： 报告是你依据真实实验填写的；ZIP 两个目录的约束没有交换；截止时间重新查提交平台，不从旧课件的相对日期推断。

第 9 步：遇到问题按这个顺序排查

目的：先检查工程和映射，再判断代码，不靠猜。

现象	优先检查
仿真输入出现 X/Z	是否选对仿真顶层；是否添加了 testbench；尝试 Relaunch；不要靠反复 Relaunch 掩盖真正的连接错误
q 在 0 ns 短暂未知	看初始复位完成后的值；未按复位前不要依赖上电状态
仿真时长不对	dff_sim 与 testbench 均为 100 ns；是否误选了辅助 check 顶层
没看到 PASS	dff_sim 本来就不打印；testbench 应打印 10 checks，出现 FATAL 才是真正检查失败
仿真成功但综合报延时不可综合	测试文件是否误放到 Design Sources，设计顶层是否误设为 testbench
未约束端口 / I/O 标准错误	是否加了对应 XDC；是否拼错顶层端口；不要降低 DRC 严重级别
时钟或器件错误	必须是 EGO1、xc7a35tcsg324-1、P17、10 ns，不能使用 Minisys 的 Y18
板卡连接不到	供电、数据线、USB-UART/JTAG 接口、实验室驱动、Auto Connect；必要时请老师协助
仍像 D 触发器或 Lab1 功能	Program Device 是否选中了本次 reg8file.bit，而不是旧 bit
清零后一松手又亮	SW7 是否仍然打开；en=1 会在下一上升沿重新写入
写了但灯不变	q 由 rsel 决定；读地址是否仍为 R0；写入时 SW7 是否真的打开过
数值前后颠倒	按 LD2_7 到 LD2_0、DIP7 到 DIP0 读；按丝印识别位号，不按外壳编号猜
写别的地址后旧值被破坏	改地址和数据前是否先关闭 SW7
按键后程序消失	是否误按 PROG；回到 Hardware Manager 重新下载正确 bit

时序与物理限制别混淆

手动开关、按键与 100 MHz 时钟没有固定相位。当前代码按课程要求直接接入，未加入去抖、输入同步器或复位释放同步器。操作时必须在关闭写使能后设置稳定的数据，再开启写入；这降低演示误操作，但不等于解决生产级亚稳态问题。

XDC 给出了管脚、I/O 标准和 10 ns 内部时钟。不要为了消掉警告而给人工开关编造外部时序数值，或给全部路径加 false path。遇到时序警告需读出具体路径，并按课程/教师要求处理。

检查点： 保留实际错误消息和现象；本地逻辑仿真通过并不保证 Vivado 的器件检查、综合、布局布线或板上验证通过。

第 10 步：本地复验与资料依据

目的：以后改代码时能重复验证，也能追溯引脚和要求。

从仓库根目录运行，不需要联网：

```
sh lab2/code/run_checks.sh
```

需要本机已有 iverilog、vvp、tclsh、python3。脚本自动创建 lab2/code/build/，错误即停止。两个 VCD 分别是 dff_sim.vcd 和 reg8file_sim.vcd，都位于 build 中；可使用已有波形查看器打开，但报告按课程要求在 Vivado 中获取实际截图。

2026-10-20 本机实际验证记录

检查	已运行结果
D 触发器自检	16 组状态/输入组合，65 次检查通过
寄存器文件自检	8 个地址 × 256 种数据；2,058 次写入，22,617 次检查通过
报告用 testbench	10 次检查通过，100 ns 结束
两份 XDC	5 / 25 个端口；管脚无重复；LVCMOS33；10 ns 时钟
两份 VCD	核对输入与输出转换时刻，100 ns 结束

Icarus 关于 @* 对全部 8 个数组元素敏感的 warning 属于预期行为。**尚未执行 Vivado 综合、实现或实物验证。** Tcl 约束检查不是 Vivado 约束求值，也不检查器件数据库。自检覆盖地址/数据组合，不是所有存储状态的形式化证明。

本地资料

• 实验内容及提交要求：2026-10-20 重新下载的官网快照。

• 实验原理及 D 触发器官方激励。

• D 触发器波形分析示例。

• 课程 Verilog 代码规范。

• Lab2 课件：用户提供，未修改；课件要求课后提交参照在线指导书。

• EGO1 硬件手册：2018.04 ver2.2；PDF 第 4 页时钟、第 6 页按键、第 7 页开关、第 8 页 LED。上述页数为 PDF 阅读器页码，不是页脚印刷页码。

• 代码入口：文件清单、功能说明、引脚与验证边界。

网站镜像默认不进 Git，换电脑后可能需要用已有下载脚本重新获取。以上板卡资料仅针对 EGO1；如果实物丝印与映射不同，暂停上板并确认板卡版本，不能换用 Minisys 引脚。

完成标准： 代码自检只是准备工作；真正完成实验仍需按前面步骤取得自己的波形、电路图和三张实物照片，并提交经过检查的报告。